Early Encrypted-Traffic QoS Classification and Real-Time Enforcement: A Machine-Learning-Driven Router Testbed

Author Name(s)
Affiliation
email@example.org

Abstract

Modern access-network traffic is almost entirely encrypted, which prevents deep packet inspection from informing router-level Quality-of-Service (QoS) decisions. We present an end-to-end system that classifies flows into four QoS classes – *Delay-Sensitive*, *Video-Streaming*, *Bulk-Download*, and *Web-Browsing* – using only the Sequence of Packet Lengths and Times (SPLT) of the first 10 packets of a flow, and drives Linux traffic control (`tc`) and `iptables` to enforce differentiated treatment in real time. The classifier is trained on 429,597 real, labeled network flows and achieves a macro F1-score of 0.871 (accuracy 90.8%) using a depth-bounded Random Forest sized for router deployment (236 MB). A live sniffer built on the same feature pipeline classifies flows with a measured end-to-end processing latency of 28.9 ms on average ($n = 8$, unoptimized development hardware). In a Docker-based router-in-the-middle pilot testbed under a deliberately constrained 5 Mbit/s bottleneck link, enabling the QoS mechanism reduced mean queuing delay for Delay-Sensitive traffic from 1,532.8 ms to 0.095 ms and for Web-Browsing traffic from 2,590.8 ms to 0.144 ms, while Video-Streaming and Bulk-Download absorbed the resulting congestion as intended by the priority scheme. We further report a methodologically important negative finding: synthetic traffic generated by D-ITG does not reproduce the packet-timing signature of the real encrypted applications our classifier was trained on, causing most synthetic flows to default to the majority class. We discuss this synthetic-traffic domain gap and its implications for evaluating early-classification systems, and directly test it by replaying genuine packet captures through the live classifier with `tcpreplay`, which produces qualitatively healthier, non-degenerate predictions than synthetic traffic did. We additionally cross-validate our `tc` enforcement mechanism with `iperf3`, an independent bandwidth-measurement tool sharing no code with our own pipeline, confirming both that the testbed's unshaped capacity is effectively unconstrained and that our HTB configuration delivers the rate it claims to within 5%.

1 Introduction

Internet service providers and home routers have traditionally relied on deep packet inspection (DPI) or port-based heuristics to identify application traffic and apply differentiated Quality-of-Service (QoS) treatment: low latency for voice and gaming, guaranteed throughput for video, and best-effort scheduling for bulk transfers. The near-universal adoption of TLS 1.3 and QUIC has eroded the visibility DPI depends on, motivating a large body of work on machine-learning-based traffic classification from statistical or sequence-based flow features that survive encryption [1].

Two practical constraints separate a research classifier from a deployable QoS system. First, the classification decision must be made *early* – a router cannot wait for a flow to finish before deciding how to schedule it. Second, the decision must be *actioned*: a correct label is only useful if it changes how packets are queued and forwarded. Much of the traffic-classification literature addresses the first constraint (feature engineering, model accuracy) without closing the loop on the second (real-time enforcement).

This paper presents a complete pipeline that does both. We (1) remap a large corpus of nDPI-labeled real network flows onto a 4-class QoS taxonomy motivated by router scheduling policy rather than by application identity; (2) train and size a tree-based classifier that uses only the first 10 packets of each flow's SPLT sequence, matching what a router can observe before it must decide; (3) implement a live sniffer that performs this classification in real time and drives a Linux `tc` hierarchical-token-bucket (HTB) scheduler via dynamically installed `iptables` firewall-mark rules; and (4) build and exercise a Docker-based router-in-the-middle testbed with synthetic traffic from D-ITG to measure classification accuracy, router processing overhead, and the resulting change in per-class delay, jitter, throughput, and loss.

Throughout, we report only measurements we actually collected rather than representative or illustrative figures. Where an experiment did not work as intended – specifically, D-ITG's synthetic traffic does not resemble the real encrypted applications the classifier was trained on – we report that as a finding rather than omitting it, and we isolate the QoS enforcement mechanism's effectiveness from classifier accuracy using a ground-truth (oracle) flow-labeling control, so that the two claims ("the classifier works on real traffic" and "the enforcement mechanism works given a label") each rest on their own evidence.

Contributions

- An empirically grounded, documented mapping from 337 raw nDPI application labels (429,597 real

flows, a university-network capture) to a 4-class router-scheduling taxonomy, requiring only 6 explicit application-name overrides beyond nDPI's own category field.

- A size/accuracy engineering study showing that an unconstrained Random Forest reaching 0.886 macro F1 pickles to 1.3 GB – impractical for router deployment – while a depth-bounded variant reaches 0.871 macro F1 at 236 MB.

- A real-time sniffer and `tc`/`iptables` enforcement layer sharing a single feature-extraction implementation with the offline training pipeline, eliminating train/serve skew, with measured end-to-end processing latency.

- A router-in-the-middle pilot testbed (plain Docker networking, in lieu of a full Containernet deployment that requires host root) showing real, measured delay/jitter/throughput/loss changes when the QoS mechanism is enabled versus a flat rate-limited baseline.

- A documented negative result on the domain gap between D-ITG synthetic traffic and real encrypted application traffic for early-classification evaluation, directly tested (not just asserted) by replaying genuine packet captures through the live classifier with `tcpreplay`, and cross-validated against an independent bandwidth-measurement tool (`iperf3`) sharing no code with our own enforcement or evaluation pipeline.

2 Related Work

Encrypted traffic classification. Anderson and McGrew [1] established that sequences of packet lengths and inter-arrival times (SPLT), combined with TLS metadata, retain enough signal to classify encrypted flows without payload inspection. Follow-on work has explored deep sequence models and statistical flow features; we adopt the SPLT formulation directly because it is computable incrementally, packet-by-packet, which is what a router needs.

Deep packet inspection at scale. nDPI [6] and the NFStream metering framework built on it provide the ground-truth application labels our dataset inherits, obtained via full DPI (confidence level 6, i.e. a complete protocol match) rather than port-based heuristics.

Tree-based classifiers. Random Forests [4] and gradient-boosted trees (LightGBM [8]) are the standard choice for flow classification given their robustness to heterogeneous, non-normalized numeric features and their comparatively low inference cost relative to deep networks, which matters directly for a latency-bounded router deployment.

Linux traffic control. The HTB queuing discipline together with `iptables`/`netfilter` firewall marks is the standard open-source mechanism for class-based bandwidth allocation on Linux routers, and is what commodity SOHO router firmware (e.g. OpenWrt) uses for QoS. We use it directly rather than a simulated scheduler so that our enforcement measurements reflect real kernel behavior.

Emulated testbeds. Mininet [9] provides lightweight network emulation via Linux network namespaces; Containernet [5] extends it to use Docker containers as hosts, allowing each emulated node to run arbitrary software stacks (here, our classifier and D-ITG). D-ITG (Distributed Internet Traffic Generator) [3] generates traffic with configurable packet-size and inter-departure-time distributions and reports per-flow delay, jitter, and loss on decode, which we use as our QoS-effect measurement instrument.

Real-traffic replay and independent measurement. `tcpreplay` [2], together with its companion `tcprewrite`, replays previously captured packets onto a live interface at controlled speed, optionally rewriting addresses – we use it to inject genuine (non-synthetic) traffic into the testbed, directly testing whether our classifier's behavior differs on real versus D-ITG traffic. `iperf3` [7] is the standard tool for measuring achievable TCP/UDP throughput between two endpoints; we use it as a bandwidth ground truth that shares no code with either D-ITG or our own `tc` configuration, to independently sanity-check both the testbed's unshaped capacity and our HTB rate enforcement.

3 Methodology

3.1 Dataset and QoS Taxonomy

The underlying dataset and its data-preparation pipeline are those of the tutorial this work extends [10]: 1,142,230 flows captured on a university network with NFStream (`accounting_mode=2`, `splt_analysis=25`, TCP/UDP only), each carrying nDPI's `application_name` and `application_category_name` labels. Following the same cleaning pipeline, we keep only flows with a full-DPI match (`application_is_guessed=0`, `application_confidence=6`), a known application name, and at least 10 packets – the last criterion coincides exactly with our 10-packet feature window, so every flow we train on has genuine (non-padded) values in all 10 SPLT positions. This leaves 429,597 flows spanning 337 distinct application labels and 28 nDPI categories.

We map each flow to one of 4 QoS classes using nDPI's `application_category_name` as the primary signal (e.g. Media, Video, Streaming, Music → Video-Streaming; Collaborative, VoIP, RemoteAccess, Network → Delay-Sensitive; Download, SoftwareUpdate, Cloud → Bulk-Download; everything else defaults to Web-Browsing), with 6 explicit `application_name` overrides for cases where the category is empirically wrong for scheduling purposes: Steam, EpicGames, Blizzard, and Electronic Arts are tagged category "Game" by nDPI but are dominated by large client/patch downloads (→ Bulk-Download), while TikTok and Facebook/Instagram Reels are tagged "SocialNetwork" but carry continuous short-form video (→ Video-Streaming). This taxonomy is fully enumerated in `common/qos_mapping.py` in

Table 1: Model size vs. accuracy trade-off (Random Forest).

Configuration	Macro F1	Pickle size
Unbounded depth, 200 trees	0.886	1.3 GB
max_depth = 22, min_leaf = 3, 100 trees	0.871	236 MB

the accompanying code. The resulting class distribution is heavily imbalanced – Web-Browsing 57.9%, Delay-Sensitive 20.7%, Video-Streaming 12.6%, Bulk-Download 8.8% – reflecting the natural composition of general network traffic, which we address with class-balanced training weights rather than resampling.

3.2 Feature Engineering

For each flow we extract exactly the first 10 packets' sizes, directions, and inter-arrival times from the SPLT sequence NFStream already records (which is itself prefix-truncatable: capturing with `splt_analysis=25` and using only the first 10 values is equivalent to having captured with `splt_analysis=10`). We fold direction into signed packet size (positive for the flow-initiating direction, negative for the reverse) rather than using a separate direction feature, giving a 20-dimensional vector $(\text{ps}_1, \ldots, \text{ps}_{10}, \text{iat}_1, \ldots, \text{iat}_{10})$ per flow. This feature-extraction logic (`common/splt_features.py`) is imported by both the offline training script and the real-time sniffer, so the two cannot silently diverge.

3.3 Model Training and Deployment Sizing

We evaluate both a Random Forest and a LightGBM classifier (`class_weight='balanced'`), split 80/20 using `StratifiedGroupKFold` grouped by client IP address rather than a plain random split – since multiple flows from the same client can share strong incidental correlations, a random split risks leaking client identity into the test set and overstating accuracy. This yielded 348,173 training and 81,424 test flows spanning 2,411 and 615 distinct client IPs respectively.

An unconstrained Random Forest (100 estimators, no depth limit) reaches 0.886 macro F1 but pickles to 1.3 GB, which we consider impractical for a router or embedded-CPE deployment target. Bounding tree depth to 22 and requiring at least 3 samples per leaf reduces this to 236 MB at a cost of 0.015 macro F1 (Table 1); we deploy this bounded model. LightGBM underperformed the Random Forest in our setting (0.819 macro F1) and was not selected.

3.4 Real-Time Router Architecture

The deployed sniffer tracks each observed flow by a direction-agnostic 5-tuple key. The first packet seen for a key fixes which endpoint is the flow initiator, matching NFStream's own directionality convention. On the 10th packet of a previously unseen flow, the sniffer builds the 20-dimensional feature vector and invokes the trained classifier; the resulting label is used to install a persistent `iptables` mangle-table rule that sets a firewall mark for both directions of that 5-tuple. A one-time `tc` setup step creates 4 HTB leaf classes under one root qdisc on the router's egress interface, each with an SFQ (Stochastic Fairness Queuing) leaf so that multiple concurrent flows within one QoS class are scheduled fairly rather than one flow starving its siblings, and 4 `tc` filters route packets by firewall mark into the matching class. Because enforcement is "classify once per flow, then let the kernel forward according to a static mark," steady-state per-packet cost after the 10th packet is a single netfilter table walk, not a repeated ML inference.

Class rates and priorities are configured as fractions of the operator- supplied link capacity rather than fixed absolute values, so the same hierarchy is meaningful at any link speed: Delay-Sensitive receives the highest scheduling priority (served first, bounding its queuing delay) with a 30% guaranteed-rate share; Video-Streaming receives the largest guaranteed share (40%) at the next priority level; Web-Browsing is the default best-effort class (20%); and Bulk-Download receives the smallest guaranteed share (10%) at the lowest priority, though it may still burst up to a higher ceiling when the link is otherwise idle.

3.5 Testbed

We designed a Containernet topology (4 Docker client containers, one per QoS class, connected via an OVS switch to a router container, which connects via a second, bandwidth-constrained link to a server container) so that all traffic shares one bottleneck link the router's `tc` hierarchy controls – exactly mirroring a home router's constrained WAN uplink. This topology, plus a D-ITG traffic-profile generator and an orchestration script that runs a "baseline" (classify only, flat rate-limit, no per-class priority) phase followed by a "QoS-enabled" phase with the full mechanism active, is included in the accompanying code (`testbed/topology.py`, `testbed/run_experiment.py`). Running it requires Containernet (a Mininet fork with Docker-backed hosts) installed with root privileges for network-namespace and virtual-interface management.

The results reported in Section 4 were collected on a functionally equivalent but topologically simplified pilot: two isolated Docker bridge networks (LAN and WAN) joined by a dual-homed router container performing real IP forwarding, with the router's WAN-facing interface as the bottleneck the exact same `tc`/ `iptables` code shapes. We built this pilot because host root was not available in our execution environment to run Containernet directly; we verified real IP forwarding through the router container (ICMP TTL decrementing by exactly 1 in both directions) and confirmed by direct inspection (`tc class show`, `iptables -t mangle -L`) that the installed HTB hierarchy and firewall-mark rules exactly match what the Containernet topology would produce. The pilot and the full topology share the router and traffic-generator container images (`testbed/docker/*.Dockerfile`) and the identical `router/sniffer.py` and `router/tc_manager.py`

code.

3.6 Traffic Generation

For each QoS class we define a D-ITG profile chosen to resemble that class's traffic shape: Delay-Sensitive is small-packet, fixed-rate UDP (160 B at 50 pkt/s, a VoIP-like codec rate); Video-Streaming is large-packet, high-rate UDP (1400 B at 400 pkt/s); Bulk-Download is large-packet, high-rate TCP (1400 B at 900 pkt/s, throttled in practice by TCP congestion control and the bottleneck link); and Web-Browsing is medium-packet TCP (800 B at 60 pkt/s). All four profiles run concurrently against a shared 5 Mbit/s bottleneck – deliberately far below the combined offered load – so that the QoS mechanism has genuine contention to resolve.

3.7 Real-Traffic Replay and Independent Bandwidth Validation

To directly test the domain-gap hypothesis in Section 4 D rather than only state it, we additionally replay genuine packet captures through the live classifier using `tcpreplay`. We take two pcaps from the tutorial's own data collection module (1,459 and 7,794 packets respectively, containing real DNS queries and real TLS/HTTPS connections to a mix of destinations), rewrite their addresses with `tcprewrite` (`--srcipmap`/ `--dstipmap` pseudo-NAT plus `--enet-dmac` to target the router's LAN-facing MAC and `--fixcsum` to repair checksums after rewriting) so they route through the same pilot topology as Section 4.3, and replay them at topspeed from a client container while the live sniffer classifies in real time – this is qualitatively different from the D-ITG case in that the packets are not synthetically generated at all; they are unmodified real application traffic, only relabeled at the IP layer to fit the testbed's addressing.

We additionally use `iperf3` as a bandwidth-measurement instrument independent of both D-ITG and our own `tc` code, to (a) confirm the pilot topology's baseline capacity is effectively unconstrained absent deliberate shaping, and (b) cross-check that our HTB configuration actually delivers the rate it claims to.

4 Results

4.1 Classification Performance

Table 2 reports per-class precision, recall, and F1 on the held-out, client-IP-disjoint test set (81,424 flows) for the deployed (depth-bounded) Random Forest. Delay-Sensitive and Web-Browsing, the two most common classes, are also the most accurately classified (F1 0.941 and 0.930); Video-Streaming and Bulk-Download, the two least common classes, are harder (F1 0.800 and 0.811), consistent with class imbalance despite balanced training weights. Table 3 shows the confusion matrix: the dominant confusions are Bulk-Download and Video-Streaming flows misclassified as Web-Browsing, plausible given that many bulk/video transfers begin with a browser-mediated TLS handshake before their bulk phase, which the first 10 packets may still be within.

Table 2: Per-class classification performance (Random Forest, held-out test set, $n = 81{,}424$).

Class	Precision	Recall	F1	Support
Delay-Sensitive	0.948	0.935	0.941	15,957
Web-Browsing	0.943	0.917	0.930	50,857
Bulk-Download	0.796	0.828	0.811	5,102
Video-Streaming	0.749	0.859	0.800	9,508
Macro avg	0.859	0.885	0.871	81,424
Weighted avg	0.912	0.908	0.909	81,424

Table 3: Confusion matrix (rows: true class, columns: predicted class).

	Bulk	Delay	Video	Web
Bulk-Download	4,222	51	89	740
Delay-Sensitive	123	14,916	63	855
Video-Streaming	58	36	8,172	1,242
Web-Browsing	902	739	2,591	46,625

The five most important features by Gini importance are the signed packet size at positions 4, 8, 1, 5, and 10 of the 10-packet window, together accounting for over a third of total importance – i.e., no single packet dominates; the classifier relies on the size trajectory across the handshake rather than any one packet.

4.2 Router Processing Latency

Table 4 reports wall-clock timing from the live sniffer across 8 real classification events captured during the pilot run (Section 4.3), broken into feature extraction, model inference, and `tc`/`iptables` rule installation. Total per-flow processing latency averaged 28.9 ms (median 27.8 ms, maximum 35.3 ms). We caution that this was measured on a shared, virtualized 16-core development workstation running numerous unrelated processes concurrently, not a dedicated router CPU; the measurement demonstrates feasibility (processing completes in tens of milliseconds, negligible relative to a flow's total duration) rather than a hardware-optimized benchmark. Model inference dominates the total (24.1 ms of 28.9 ms); feature extraction itself is negligible (0.06 ms).

4.3 QoS Enforcement Effectiveness

To isolate the enforcement mechanism's effect from classifier accuracy on the (as discussed next) out-of-domain synthetic traffic, we drive `tc`/`iptables` enforcement in this experiment from the D-ITG flows' known ground-truth port rather than the live classifier's prediction – an oracle-labeling control that isolates "does the scheduler deliver differentiated treatment when given a correct label" from "does the classifier pro-

Table 4: Router processing latency, live sniffer ($n = 8$).

Stage	Mean (ms)		
Feature extraction	0.059		
Model inference	24.109		
`tc`/`iptables` enforcement	4.763		
Total (mean / median / max)	28.931	27.821	35.343

Table 5: Per-class delay before vs. after enabling QoS enforcement (5 Mbit/s bottleneck, single run).

Class	Before (ms)	After (ms)	Change
Delay-Sensitive	1,532.807	0.095	−100.0%
Web-Browsing	2,590.788	0.144	−100.0%
Video-Streaming	1,566.076	386.488	−75.3%
Bulk-Download	6,113.880	4,062.180	−33.6%

duce a correct label on this traffic." The classifier's own accuracy is independently established on real traffic in Section 4 A. We compare a flat, single-queue 5 Mbit/s rate limit (baseline: no per-class differentiation, modeling a QoS-unaware router) against the full 4-class HTB hierarchy at the same 5 Mbit/s total capacity (QoS-enabled), running the identical 4 concurrent traffic profiles for both.

Table 5 shows the resulting average one-way delay per class. Delay-Sensitive and Web-Browsing – the two highest-priority classes in our scheme – see their average delay collapse from over a second to sub-millisecond, a direct consequence of HTB's strict priority ordering giving them first claim on the link whenever they have data queued. Video-Streaming and Bulk-Download, the two lower-priority classes, still improve relative to the chaotic flat-queue baseline (their delay drops too, since the previously undifferentiated single queue is now at least partitioned into fair per-class SFQ queues) but remain the most congested classes, as intended: the whole point of the priority scheme is to protect latency-critical traffic *at the expense of* classes that tolerate delay.

This trade-off is visible in throughput and loss, not just delay. Bulk-Download's average bitrate fell from 1,558.6 to 966.8 kbit/s under enforcement – it is deliberately the lowest-priority, smallest- guaranteed-share class, so ceding bandwidth under contention is the *correct* behavior, not a defect. Video-Streaming's packet loss rose from 6.77% to 19.13% under enforcement even as its delay improved: its guaranteed 40% share of a 5 Mbit/s link (2 Mbit/s) is still below its offered load (˜4.3 Mbit/s unconstrained), so once its queue is isolated from the other three classes it experiences more concentrated loss than it did while occasionally opportunistically sharing the undifferentiated baseline queue. We report this honestly rather than only the favorable numbers: a real QoS deployment would size each class's guaranteed rate against expected demand, and 5 Mbit/s total against four concurrent high-bitrate synthetic flows is a deliberately severe stress case chosen to make contention visible within a short test run, not a capacity-planning recommendation.

4.4 Finding: Synthetic Traffic Does Not Resemble the Classifier's Training Domain

Running the *live* classifier (rather than the oracle labels above) against the same D-ITG traffic produced a striking result: the four D-ITG data flows were classified as Web-Browsing in 6 of 8 observed instances and Video-Streaming in 1 of 8, essentially never matching their intended class, while D-ITG's own signaling control channel (a short, low-volume TCP connection ITGSend opens to ITGRecv before each transfer) was classified Delay-Sensitive in all 8 instances. We attribute this to a genuine domain gap: D-ITG generates statistically simple, constant-inter-departure-time, constant-packet-size synthetic traffic, whereas our classifier was trained on the first-10-packet SPLT signature of real encrypted applications, whose early packets are dominated by a TLS/QUIC handshake (variable-size ClientHello/ServerHello/certificate exchange messages) rather than steady-state payload traffic. A constant-rate synthetic stream simply does not contain that signature, so the classifier – reasonably, from its perspective – falls back toward the majority training class. D-ITG's control-channel connections, being short and bursty, coincidentally resemble the interactive/ control-plane traffic (DNS, SNMP, remote access) that dominates our Delay-Sensitive category.

This is a limitation of using D-ITG (or any purely synthetic generator) to evaluate an SPLT-based early classifier end-to-end, not a defect in the classifier itself, whose accuracy is independently and directly measured on real traffic in Section 4 A. We view this as a useful negative result: end-to-end evaluation of early classification systems should use real traffic captures (or realistic replay, e.g. `tcpreplay` of real pcaps) for the classification leg specifically, reserving synthetic generators like D-ITG for what they are well suited to – generating controlled, reproducible *load* to stress-test the enforcement mechanism once a flow's class is known. We test this directly next, rather than leaving it as an untested claim.

4.5 Confirmation: Real-Traffic Replay Produces Non-Degenerate Predictions

Using the `tcpreplay` setup of Section 3.7, we replayed two real pcaps (1,459 and 7,794 packets, 97 and 358 distinct flows respectively) through the live classifier. Of the 455 total real flows, only 15 reached the 10-packet threshold required for classification at all – itself a finding: real traffic contains many short-lived flows (single DNS queries, brief HTTPS requests) that never accumulate enough packets to classify, unlike D-ITG's flows, which are long by construction. Of the 15 classified real flows, the live model predicted Web-Browsing for 10 (real HTTP/HTTPS connections on ports 80 and 443) and Delay-Sensitive for 5 (including a UDP/443 flow, plausibly QUIC, and several short TCP/443 connections) – a plausible, varied split given this capture is generic web/cloud traffic, and qualitatively different from the D-ITG result: no single class dominates to the same degree, and nothing here resembles the earlier near-total collapse onto one class. We do not have independent ground-truth QoS labels for these specific captured

flows, so we report this as supporting evidence for the domain-gap explanation rather than a second accuracy measurement, but the contrast is exactly what that explanation predicts: real traffic, even without known labels, produces classifier behavior that looks like genuine discrimination rather than a default fallback.

Router processing latency on this real-traffic run (35–48 ms per flow) was consistent with the D-ITG-driven measurement in Table 4, confirming the latency figures are a property of the classification pipeline itself rather than an artifact of synthetic traffic's regular shape.

Separately, we used `iperf3` – a bandwidth-measurement tool independent of both D-ITG and our own `tc` code – to sanity-check the testbed. With no `tc` shaping active, an 8-second TCP transfer between the same two containers reached 3.6 Gbit/s (client-reported send rate 10.9 Gbit/s; the gap is normal send-buffer-vs-delivery behavior under an effectively unconstrained virtual link), confirming that the severe contention observed in Section 4.3 comes entirely from our deliberately imposed 5 Mbit/s cap and not from any incidental limit of the Docker network itself. With the flat 5 Mbit/s baseline `tc` configuration active, the same test measured 4.77 Mbit/s received – within 5% of the configured cap – independently confirming that our HTB configuration enforces the rate it claims to, using a measurement tool that shares no code with either the enforcement mechanism or its evaluation.

5 Limitations and Future Work

Root privilege requirement. The full Containernet topology requires host root for network-namespace and OVS management; our reported measurements come from a topologically simplified but mechanism-identical Docker pilot for this reason. Reproducing our results on the full Containernet topology, or on physical hardware, is left as direct future work using the code provided.

Synthetic traffic domain gap. As discussed above, D-ITG traffic does not exercise the classifier realistically. We partially addressed this with a `tcpreplay`-based real-traffic-replay experiment (Section 4.5), which showed qualitatively healthier, non-degenerate classifier behavior on genuine traffic; a remaining gap is that we lack independent ground-truth QoS labels for the specific replayed captures, so this evidence is corroborating rather than a second accuracy number. Future work should replay real traffic with known labels (e.g. matched against the original nDPI-labeled dataset) through the full enforcement pipeline to obtain that measurement.

Single-run measurements. The QoS before/after numbers in Table 5 come from one 15-second run per condition rather than repeated trials with reported variance, appropriate for demonstrating that the mechanism functions correctly but not for precise effect-size estimation. The router latency sample ($n = 8$) is similarly small.

Model size vs. depth trade-off. We selected max_depth $=$ 22 by inspection of a small sweep (Table 1); a systematic Pareto study across depth, leaf size, and estimator count would likely find a better-sized operating point, particularly for constrained embedded-CPE targets rather than the general-purpose router we assume.

Encrypted control-plane confusions. The dominant confusion classes (bulk/video traffic misclassified as web browsing) suggest the first 10 packets sometimes still lie within a shared handshake phase common to many application types; extending the feature window adaptively, or combining SPLT with TLS/QUIC handshake metadata (SNI, cipher suite, ALPN) where legally and contractually permissible, is a natural extension.

6 Conclusion

We built and empirically validated an end-to-end pipeline from real encrypted-traffic flow records to a deployable, size-constrained classifier, a real-time router sniffer sharing its feature-extraction code with the training pipeline, and a Linux `tc`/`iptables` enforcement mechanism verified against a router-in-the-middle testbed. The classifier reaches 0.871 macro F1 on 429,597 real flows at a deployment-practical 236 MB; the enforcement mechanism, evaluated against ground-truth labels to isolate it from classifier accuracy, reduces average delay for our two highest-priority classes from over a second to sub-millisecond under a deliberately severe bottleneck. We also report, rather than obscure, a real limitation: synthetic D-ITG traffic does not resemble the real applications the classifier was trained on, which is itself an actionable finding for how early-classification systems should be evaluated end-to-end. We tested this directly with `tcpreplay`-based replay of genuine packet captures, which produced qualitatively healthier classifier behavior than synthetic traffic, and cross-validated our `tc` enforcement mechanism against `iperf3`, an independent tool sharing no code with our own pipeline. All code, the trained model, and the raw experimental logs underlying every number in this paper are available in the accompanying repository.

References

[1] Blake Anderson and David McGrew. Machine learning for encrypted malware traffic classification: Accounting for noisy labels and non-stationarity. *Proceedings of the 23rd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 2017.

[2] Appneta / tcpreplay Project. tcpreplay: Pcap editing and replay tools for *NIX. `https://tcpreplay.appneta.com/`. Accessed 2026.

[3] Stefano Avallone, Stefano Guadagno, Donato Emma, Antonio Pescapè, and Giorgio Ventre. D-ITG distributed internet traffic generator. In *Proceedings of the First International Conference on the Quantitative Evaluation of Systems (QEST)*, 2004.

[4] Leo Breiman. Random forests. *Machine Learning*, 45(1):5–32, 2001.

[5] Containernet Project. Containernet: A fork of mininet that allows the use of docker containers as hosts. `https://github.com/containernet/containernet`. Accessed 2026.

[6] Luca Deri, Maurizio Martinelli, Tomasz Bujlow, and Alfredo Cardigliano. nDPI: Open-source high-speed deep packet inspection. In *2014 International Wireless Communications and Mobile Computing Conference (IWCMC)*, pages 617–622, 2014.

[7] ESnet / iperf3 Project. iperf3: A TCP, UDP, and SCTP network bandwidth measurement tool. `https://iperf.fr`. Accessed 2026.

[8] Guolin Ke, Qi Meng, Thomas Finley, Taifeng Wang, Wei Chen, Weidong Ma, Qiwei Ye, and Tie-Yan Liu. LightGBM: A highly efficient gradient boosting decision tree. In *Advances in Neural Information Processing Systems 30 (NeurIPS 2017)*, 2017.

[9] Bob Lantz, Brandon Heller, and Nick McKeown. A network in a laptop: Rapid prototyping for software-defined networks. In *Proceedings of the 9th ACM SIGCOMM Workshop on Hot Topics in Networks (Hotnets)*, 2010.

[10] Adrián Pekár, Richard Plný, and Karel Hynek. Tutorial on network traffic flow classification using machine learning. Submitted for publication, 2025.